پاسخ سوال یک:

برای قابلیت بازگرداندن آخرین تغییرات (Undo) ، ساختار Stack مناسب است، زیرا پشته بر اساس اصل LIFO عمل می‌کند؛ یعنی آخرین تغییری که انجام شده، اولین تغییری است که باید بازگردانده شود. برای مدیریت درخواست‌های ویرایشی به ترتیب ورود، ساختار Queueانتخاب مناسبی است، زیرا صف از اصل FIFO پیروی می‌کند و در نتیجه درخواستی که زودتر ثبت شده، زودتر پردازش می‌شود.
همچنین برای نگهداری پاراگراف‌ها یا بخش‌های مختلف کتاب می‌توان از Linked List استفاده کرد، زیرا هر بخش به بخش بعدی متصل است و اضافه کردن، حذف یا جابه‌جایی بخش‌ها بدون نیاز به جابه‌جایی تمام داده‌ها امکان‌پذیر است. بنابراین هر یک از این سه ساختار داده با توجه به ویژگی خاص خود می‌تواند بخشی از عملکرد ویرایشگر متن را به شکل مناسبی مدیریت کند.

پاسخ سوال دو:

در این مسئله یک لیست پیوندی یک‌طرفه در اختیار داریم و هدف این است که هر مقدار جدید به انتهای لیست اضافه شود،

بدون اینکه ترتیب گره‌های قبلی تغییر کند. برای انجام این کار، ابتدا با استفاده از مقدار ورودی data یک گره جدید ساخته می‌شود.

این گره در ابتدا به هیچ گره دیگری متصل نیست، بنابراین اشاره‌گر next آن برابر None قرار می‌گیرد. سپس باید وضعیت لیست بررسی شود. اگر head برابر None باشد، به این معناست که لیست خالی است و هیچ گره‌ای در آن وجود ندارد. در این حالت، گره جدید به عنوان اولین و تنها گره لیست در نظر گرفته می‌شود و همان گره به عنوان سر لیست بازگردانده می‌شود.

اگر لیست خالی نباشد، باید گره جدید در انتهای لیست قرار گیرد. برای پیدا کردن آخرین گره، پیمایش از سر لیست آغاز می‌شود و با استفاده از اشاره‌گر next از هر گره به گره بعدی حرکت می‌کنیم. این روند تا زمانی ادامه پیدا می‌کند که به گره‌ای برسیم که اشاره‌گر next آن برابر None باشد. این گره آخرین گره فعلی لیست است. در این مرحله، اشاره‌گر next آخرین گره به گره جدید متصل می‌شود. از آنجا که اشاره‌گر next گره جدید همچنان برابر None است، گره جدید به آخرین عضو لیست تبدیل می‌شود.

در پایان، همان سر اولیه لیست بازگردانده می‌شود، زیرا در حالت غیرخالی بودن لیست، گره ابتدایی تغییری نکرده است و فقط ارتباط آخرین گره تغییر کرده است. به این ترتیب، ترتیب تمام گره‌های قبلی حفظ می‌شود و مقدار جدید در انتهای لیست قرار می‌گیرد.

برای مثال، اگر لیست اولیه به صورت زیر باشد:

10 ⟷ 20 ⟷ 30

و مقدار ۴۰ به تابع داده شود، پیمایش از گره ۱۰ شروع می‌شود، سپس به گره ۲۰ و بعد به گره ۳۰ می‌رسیم. چون گره ۳۰ آخرین گره است، گره جدید با مقدار ۴۰ به آن متصل می‌شود و لیست نهایی به شکل زیر خواهد بود:

10 ⟷ 20 ⟷ 30 ⟷ 40

از نظر پیچیدگی زمانی، اگر لیست دارای n گره باشد، در بدترین حالت باید تمام گره‌ها پیمایش شوند تا آخرین گره پیدا شود. بنابراین زمان اجرای این روش برابر O(n) است. همچنین برای انجام عملیات، به جز یک گره جدید و یک اشاره‌گر موقت، فضای اضافی دیگری موردنیاز نیست؛ بنابراین پیچیدگی فضای کمکی برابر O(۱) است.

پاسخ سوال سه:

در این مسئله هدف این است که رفتار یک صف با استفاده از دو پشته شبیه سازی شود. در صف، عناصر بر اساس اصل «اولین ورودی، اولین خروجی» پردازش می شوند؛ یعنی عنصری که زودتر وارد شده است باید زودتر نیز خارج شود. در مقابل، پشته بر اساس اصل «آخرین ورودی، اولین خروجی» عمل می کند؛ یعنی آخرین عنصری که وارد پشته شده است، نخستین عنصری خواهد بود که از آن خارج می شود.

به همین دلیل، برای ساختن رفتار یک صف با استفاده از پشته ها، لازم است از دو پشته استفاده کنیم تا ترتیب عناصر در زمان مناسب معکوس شود.

در این پیاده سازی دو پشته با نام های stack1 و stack2 در نظر گرفته می شوند. وظیفه stack1 نگهداری عناصر جدیدی است که وارد صف می شوند. هر زمان عملیات enqueue انجام شود، عنصر جدید مستقیما به انتهای stack1 اضافه می شود. این عملیات ساده است و نیازی به جابه جایی سایر عناصر ندارد.

برای مثال، اگر عملیات های زیر به ترتیب انجام شوند:

enqueue 10

enqueue 20

enqueue 30

وضعیت دو پشته به صورت زیر خواهد بود:

stack1 = [10, 20, 30]

stack2 = []

در این حالت عدد ۳۰ در بالای stack1 قرار دارد. با این حال، اگر بخواهیم یک عنصر را از صف خارج کنیم، باید عدد ۱۰ حذف شود، زیرا این مقدار زودتر از سایر عناصر وارد شده است. بنابراین نمی توان مستقیما از stack1 عنصر را حذف کرد.

برای حل این مشکل، هنگامی که عملیات dequeue یا front انجام می شود، ابتدا وضعیت stack2 بررسی می شود. اگر stack2 خالی باشد، تمام عناصر موجود در stack1 یکی یکی از آن خارج شده و وارد stack2می شوند. چون عمل انتقال با استفاده از رفتار پشته انجام می شود، ترتیب عناصر معکوس خواهد شد.

در مثال قبل، پس از انتقال عناصر خواهیم داشت:

stack1 = []

stack2 = [30, 20, 10]

اکنون عدد ۱۰ در بالای stack2 قرار گرفته است. این دقیقا همان عنصری است که در یک صف باید زودتر از سایر عناصر خارج شود. بنابراین با انجام عملیاتdequeue ، عنصر بالای stack2 حذف شده و مقدار ۱۰ برگردانده می شود.

پس از این عملیات، وضعیت stack2 به صورت زیر خواهد بود:

stack2 = [30, 20]

در نتیجه اگر عملیات بعدی dequeue انجام شود، مقدار ۲۰ خارج خواهد شد و اگر عملیات front انجام شود، مقدار ۲۰ بدون حذف شدن مشاهده می شود.

نکته مهم این است که عناصر stack1 نباید در هر بار اجرای dequeueیا front دوباره به stack2 منتقل شوند. انتقال تنها زمانی انجام می شود که stack2 خالی باشد.

تا زمانی که stack2 دارای عنصر است، قدیمی ترین عناصر صف به ترتیب صحیح در آن قرار دارند و می توان مستقیما از آن استفاده کرد. این موضوع باعث می شود از انجام جابه جایی های غیرضروری جلوگیری شود.

اگر پس از انتقال اولیه، عنصر جدیدی وارد صف شود، آن عنصر دوباره در stack1 قرار می گیرد. برای مثال، فرض کنیم پس از وضعیت زیر:

stack1 = []

stack2 = [30, 20]

عملیات زیر انجام شود:

enqueue 40

در این صورت خواهیم داشت:

stack1 = [40]

stack2 = [30, 20]

در این حالت هنوز نباید عناصر stack1 به stack2 منتقل شوند، زیرا stack2خالی نیست و عدد ۲۰ همچنان قدیمی ترین عنصر صف است. پس از حذف ۲۰ و سپس ۳۰، stack2خالی می شود. در آن زمان، اگر عملیات dequeue یا front دیگری انجام شود، عناصر موجود در stack1به stack2 منتقل می شوند و مقدار ۴۰ در دسترس قرار می گیرد. به این ترتیب ترتیب ورود عناصر در تمام مراحل حفظ می شود.

از نظر پیچیدگی زمانی، عملیات enqueue همیشه فقط شامل اضافه کردن یک عنصر به stack1 است؛ بنابراین زمان اجرای آن برابر O(1) است. عملیات dequeue و front در بعضی مواقع ممکن است نیاز داشته باشند تمام عناصر موجود در stack1 را به stack2 منتقل کنند. اگر n عنصر در stack1 وجود داشته باشد، چنین عملیاتی در همان لحظه می تواند O(n) زمان نیاز داشته باشد. با این حال، هر عنصر پس از ورود به stack1 حداکثر یک بار از stack1 خارج و وارد stack2 می شود و پس از آن نیز یک بار از stack2 خارج خواهد شد. بنابراین هزینه این انتقال ها بین مجموعه ای از عملیات ها تقسیم می شود و هزینه سرشکن عملیات dequeue و front برابر O(1) در نظر گرفته می شود. تفاوت اصلی میان یک صف معمولی و پیاده سازی صف با استفاده از دو پشته، در نحوه دسترسی به قدیمی ترین عنصر است. در یک صف معمولی، ساختار داده از ابتدا به گونه ای طراحی شده است که عنصر جدید از یک سمت وارد و قدیمی ترین عنصر از سمت دیگر خارج شود. بنابراین عملیات افزودن و حذف بدون نیاز به معکوس کردن ترتیب عناصر انجام می شود.

اما در پیاده سازی با دو پشته، هر پشته به تنهایی دارای رفتار «آخرین ورودی، اولین خروجی» است و این رفتار با رفتار موردنیاز صف متفاوت است.

به همین دلیل از پشته اول برای دریافت عناصر جدید و از پشته دوم برای معکوس کردن ترتیب آن ها استفاده می شود. هنگامی که پشته

دوم خالی باشد، عناصر پشته اول به آن منتقل می شوند و در نتیجه عنصری که زودتر وارد شده است در بالای پشته دوم قرار میگیرد.

در یک عملیات خاص dequeue یا front ممکن است لازم باشد تعداد زیادی عنصر جابه جا شوند و در نتیجه زمان اجرای آن عملیات به O(n) برسد. با این حال، این جابه جایی در هر بار انجام نمی شود و هر عنصر در طول حضور خود در ساختار فقط تعداد محدودی بار بین پشته ها حرکت می کند. به همین دلیل اگر تعداد زیادی عملیات متوالی را با هم در نظر بگیریم، هزینه متوسط یا سرشکن هر عملیات همچنان O(1) خواهد بود. این در حالی است که در یک صف معمولی، در صورت استفاده از پیاده سازی مناسب، عملیات های اصلی صف مستقیما و بدون نیاز به این مرحله انتقال انجام می شوند.

پاسخ سوال چهار:

برای حل این مسئله، لیست پیوندی باید به بخش‌هایkتایی تقسیم شود و اتصال گره‌های هر بخش به صورت معکوس تغییر کند. ابتدا از ابتدای هر گروه بررسی می‌کنیم که آیا حداقل k گره تا انتهای لیست باقی مانده است یا خیر. اگر تعداد گره‌های باقی‌مانده کمتر از k باشد، آن بخش بدون تغییر باقی می‌ماند و عملیات تمام می‌شود.

اگر یک گروه کامل شامل k گره وجود داشته باشد، باید گره‌های همان گروه را یکی یکی پیمایش کرده و جهت اشاره‌گر next آن‌ها را معکوس کنیم. برای این کار معمولا از چند اشاره‌گر موقت استفاده می‌شود تا

هنگام تغییر next یک گره، دسترسی به ادامه لیست از بین نرود. پس از معکوس شدن یک گروه، اولین گره قبلی آن گروه به آخرین گره گروه تبدیل می‌شود و باید به ابتدای گروه بعدی متصل شود. همچنین انتهای گروه قبلی نیز باید به ابتدای جدید گروه معکوس‌شده متصل شود.

برای مثال، اگر لیست به صورت زیر باشد:

1 ← 2 ← 3 ← 4 ← 5

و مقدار k برابر ۲ باشد، ابتدا دو گره اول یعنی ۱ و ۲ بررسی و معکوس می‌شوند:

2 ← 1

سپس گروه بعدی شامل ۳ و ۴ است که به صورت زیر تغییر می‌کند:

4 ← 3

در انتها فقط گره ۵ باقی می‌ماند. چون تعداد گره‌های باقی‌مانده کمتر از ۲ است، این گره بدون تغییر باقی می‌ماند. بنابراین لیست نهایی برابر است با:

2 ← 1 ← 4 ← 3 ← 5

اگر مقدار k برابر ۳ باشد، سه گره اول معکوس می‌شوند:

1 ← 2 ← 3

به:

1 _ 2 _ 3

تبدیل می‌شود. پس از آن فقط دو گره ۴ و ۵ باقی می‌مانند و چون تعداد آن‌ها کمتر از ۳ است، ترتیبشان تغییر نمی‌کند. در نتیجه خروجی نهایی به صورت زیر خواهد بود:

5 _ 4 _ 1 _ 2 _ 3

نکته مهم این است که مقدار ذخیره‌شده داخل گره‌ها نباید جابه‌جا شود. یعنی نمی‌توان مقادیر ۱ و ۲ را با یکدیگر عوض کرد و گفت گروه معکوس شده است؛ بلکه باید خود ارتباط میان گره‌ها با تغییر اشاره‌گرهای next اصلاح شود. به همین دلیل، این مسئله بیشتر توانایی مدیریت صحیح اشاره‌گرها در لیست پیوندی را بررسی می‌کند.

برای ساده‌تر شدن اتصال گروه‌ها می‌توان از یک گره کمکی در ابتدای لیست استفاده کرد. این گره باعث می‌شود معکوس کردن اولین گروه نیز مانند سایر گروه‌ها انجام شود و نیاز به در نظر گرفتن حالت جداگانه برای تغییر head کمتر شود. پس از پایان عملیات، گره بعد از این گره کمکی همان head جدید لیست خواهد بود.

از نظر پیچیدگی زمانی، هر گره حداکثر تعداد محدودی بار بررسی می‌شود و عملیات اصلی با یک پیمایش روی لیست انجام می‌شود؛ بنابراین اگر تعداد گره‌ها n باشد، زمان اجرای الگوریتم O(n) است.

همچنین اگر معکوس‌سازی به صورت مستقیم و با اشاره‌گرهای موقت انجام شود، به فضای اضافی وابسته به تعداد گره‌ها نیازی نیست و پیچیدگی فضای کمکی O(1) خواهد بود.

پاسخ سؤال پنج :

برای حل این مسئله، پشته با استفاده از یک آرایه با ظرفیت مشخص پیاده‌سازی می‌شود. در ابتدای کار ظرفیت آرایه برابر ۴ و تعداد عناصر موجود در پشته برابر صفر است. علاوه بر آرایه اصلی، دو مقدار برای نگهداری تعداد عناصر فعلی و ظرفیت فعلی آرایه در نظر گرفته می‌شود.

در عملیات افزودن، ابتدا بررسی می‌شود که آیا آرایه فضای خالی دارد یا خیر. اگر تعداد عناصر کمتر از ظرفیت باشد، عنصر جدید مستقیماً در خانه بعدی قرار می‌گیرد و تعداد عناصر یک واحد افزایش پیدا می‌کند. اما اگر تعداد عناصر با ظرفیت برابر باشد، آرایه پر است. در این حالت ابتدا یک آرایه جدید با ظرفیتی دو برابر آرایه قبلی ساخته می‌شود. تمام عناصر آرایه قبلی یکی‌یکی و با همان ترتیب به آرایه جدید منتقل می‌شوند و پس از آن عنصر تازه به پشته اضافه می‌شود.

برای مثال، اگر ظرفیت ۴ باشد و پشته شامل عناصر زیر باشد:

10 ، ۲۰ ، ۳۰ ، ۴۰

برای اضافه کردن مقدار ۵۰ ابتدا باید ظرفیت از ۴ به ۸ افزایش پیدا کند. سپس چهار عنصر قبلی به آرایه جدید منتقل شده و مقدار ۵۰ نیز در خانه بعدی قرار می‌گیرد.

در عملیات حذف، ابتدا خالی بودن پشته بررسی می‌شود. اگر پشته خالی باشد، مقدار None برگردانده می‌شود. در غیر این صورت آخرین عنصر موجود، که همان عنصر بالای پشته است، حذف شده و تعداد عناصر یک واحد کاهش پیدا می‌کند.

پس از هر حذف، ظرفیت آرایه نیز بررسی می‌شود. اگر تعداد عناصر باقی‌مانده کمتر یا مساوی یک‌چهارم ظرفیت باشد و ظرفیت فعلی بیشتر از ۴ باشد، ظرفیت به نصف کاهش پیدا می‌کند. برای این کار نیز یک آرایه جدید ساخته شده و تمام عناصر باقی‌مانده به آن منتقل می‌شوند. ظرفیت هیچ‌گاه اجازه ندارد از ۴ کمتر شود.

برای مثال اگر ظرفیت برابر ۸ و تعداد عناصر پس از حذف برابر ۲ شود، چون:

2 = 8 / 4

ظرفیت از ۸ به ۴ کاهش پیدا می‌کند.

عملیات مشاهده عنصر بالای پشته فقط آخرین عنصر را برمی‌گرداند و تغییری در پشته ایجاد نمی‌کند. عملیات نمایش اندازه نیز تعداد عناصر واقعی موجود را برمی‌گرداند و عملیات نمایش ظرفیت، اندازه فعلی آرایه را مشخص می‌کند.

نکته اصلی این مسئله این است که افزایش و کاهش ظرفیت در هر عملیات اتفاق نمی‌افتد. ممکن است یک عملیات خاص به دلیل نیاز به انتقال عناصر هزینه زیادی داشته باشد، اما بین دو تغییر ظرفیت تعداد زیادی عملیات ساده انجام می‌شود. همین موضوع اساس تحلیل سرشکن این ساختار است.

پاسخ تکمیلی سؤال یک

در حالت عادی، اضافه کردن یک عنصر فقط شامل قرار دادن آن در خانه بعدی آرایه است و در زمان ثابت انجام می‌شود؛ بنابراین پیچیدگی آن برابر $O(1)$ است.

اما اگر آرایه پر باشد، باید تمام عناصر موجود به یک آرایه جدید با ظرفیت دو برابر منتقل شوند. اگر n عنصر وجود داشته باشد، انتقال آن‌ها از مرتبه $O(n)$ زمان می‌برد. بنابراین بدترین حالت یک عملیات افزودن برابر است با:

$O(n)$

عملیات حذف نیز معمولاً $O(1)$ است، زیرا فقط آخرین عنصر حذف می‌شود. اما اگر پس از حذف، شرط کاهش ظرفیت برقرار شود، باید عناصر باقی‌مانده به یک آرایه جدید منتقل شوند.

در این حالت نیز یک عملیات حذف خاص می‌تواند دارای پیچیدگی:

O(n)

باشد.

پس بدترین حالت هر دو عملیات ممکن است O(n) باشد، اما این هزینه در هر بار اجرا اتفاق نمی‌افتد.

پاسخ تکمیلی سوال دو

برای تحلیل سرشکن باید به جای بررسی یک عملیات خاص، مجموعه بزرگی از عملیات‌ها را با هم بررسی کنیم.

در زمان افزایش ظرفیت، اندازه آرایه به صورت زیر تغییر می‌کند:

4 = 8 = 16 = 32 = 64 = ...

در هر مرحله تعداد عناصری که باید منتقل شوند به صورت تقریبی برابر است با:

4 + 8 + 16 + 32 + ...

این مجموع یک دنباله هندسی است و تا زمانی که تعداد عناصر به n برسد، مجموع کل انتقال‌ها از مرتبه O(n) خواهد بود.

در مورد کاهش ظرفیت نیز به دلیل اینکه کاهش در یک‌چهارم ظرفیت انجام می‌شود، دو تغییر ظرفیت نمی‌توانند بلافاصله پشت سر هم رخ

دهند. بین دو تغییر اندازه باید تعداد قابل توجهی عملیات افزودن یا حذف انجام شود.

بنابراین در یک دنباله شامل n عملیات، مجموع هزینه عملیات‌های معمولی و انتقال عناصر از مرتبه:

O(n)

خواهد بود.

پس هزینه سرشکن هر عملیات برابر است با:

O(n) / n = O(1)

بنابراین با وجود اینکه بعضی عملیات‌های خاص O(n) هزینه دارند، هزینه سرشکن هر عملیات افزودن یا حذف O(1) است.

پاسخ تکمیلی سوال سه

اگر ظرفیت را زمانی نصف کنیم که تعداد عناصر به نصف ظرفیت رسیده باشد، ممکن است ساختار دائماً بین دو ظرفیت رفت‌وبرگشت کند.

برای مثال فرض کنید:

ظرفیت = ۸

تعداد عناصر = ۵

اگر یک عنصر حذف شود:

تعداد عناصر = ۴

طبق قانون فرضی، چون تعداد عناصر به نصف ظرفیت رسیده است، ظرفیت از ۸ به ۴ کاهش پیدا می‌کند.

اکنون داریم:

ظرفیت = ۴

تعداد عناصر = ۴

یعنی آرایه کاملاً پر است.

اگر فقط یک عنصر جدید اضافه کنیم، آرایه دوباره باید بزرگ شود:

8 ⁼ 4

پس از افزودن داریم:

ظرفیت = ۸

تعداد عناصر = ۵

اگر همان عنصر دوباره حذف شود، تعداد عناصر به ۴ می‌رسد و ظرفیت دوباره نصف می‌شود:

4 ⁼ 8

بنابراین با تکرار عملیات‌های:

افزودن ⁼ حذف ⁼ افزودن ⁼ حذف... ⁼

ظرفیت می‌تواند دائماً بین ۴ و ۸ تغییر کند.

هر بار تغییر ظرفیت نیز نیازمند انتقال چندین عنصر است. در نتیجه یک دنباله ساده از افزودن و حذف می‌تواند بارها باعث کپی شدن عناصر شود و عملکرد ساختار را به شدت کاهش دهد.

پاسخ تکمیلی سوال چهار

برای جلوگیری از افزایش و کاهش مکرر ظرفیت، آستانه کاهش را یک‌چهارم در نظر می‌گیریم.

فرض کنید ظرفیت فعلی برابر m باشد و تعداد عناصر پس از حذف به:

m / 4

رسیده باشد.

در این حالت ظرفیت نصف می‌شود و ظرفیت جدید برابر است با:

m / 2

اما تعداد عناصر همچنان:

m / 4

است.

بنابراین نسبت پر بودن آرایه جدید برابر است با:

(m / 4) ÷ (m / 2) = 1 / 2

یعنی پس از کاهش ظرفیت، آرایه جدید فقط نصفه پر است.

برای اینکه این آرایه دوباره کاملاً پر شود و نیاز به دو برابر شدن ظرفیت پیدا کند، باید تقریباً:

$m / 4$

عنصر جدید به آن اضافه شود.

در نتیجه بین کاهش ظرفیت و افزایش بعدی، تعداد زیادی عملیات عادی انجام می‌شود. همین فاصله باعث می‌شود هزینه انتقال عناصر بین تعداد زیادی عملیات تقسیم شود و ساختار دچار افزایش و کاهش مکرر ظرفیت نشود.

پاسخ تکمیلی سؤال پنج

در روش پتانسیل، مقداری به نام پتانسیل برای وضعیت فعلی ساختار تعریف می‌کنیم. این مقدار را می‌توان نوعی هزینه ذخیره‌شده در عملیات‌های ارزان در نظر گرفت که بعداً هزینه عملیات‌های گران را جبران می‌کند.

اگر n تعداد عناصر و m ظرفیت فعلی باشد، تابع پتانسیل به صورت زیر تعریف شده است:

اگر:

$n \geq m / 2$

باشد:

$\Phi = 2n - m$

و اگر:

$n < m / 2$

باشد:

$\Phi = m / 2 - n$

هزینه سرشکن هر عملیات از رابطه زیر به دست می‌آید:

هزینه سرشکن = هزینه واقعی + تغییر پتانسیل

در یک افزودن معمولی که نیازی به افزایش ظرفیت ندارد، هزینه واقعی برابر ۱ است و مقدار پتانسیل نیز فقط به اندازه یک مقدار ثابت تغییر می‌کند. بنابراین هزینه سرشکن همچنان $O(1)$ است.

حالت مهم زمانی است که آرایه کاملاً پر شده باشد. فرض کنیم قبل از افزودن:

$n = m$

باشد.

پتانسیل قبل از عملیات برابر است با:

Φقبل = m۲ - m = m

برای افزودن عنصر جدید، باید m عنصر قبلی کپی شده و یک عنصر جدید نیز اضافه شود.

بنابراین هزینه واقعی تقریباً برابر است با:

m + 1

بعد از دو برابر شدن ظرفیت داریم:

ظرفیت جدید = m۲

تعداد عناصر جدید:

n = m + 1

بنابراین پتانسیل جدید برابر می‌شود با:

Φبعد = ۲m - (m + 1)

در نتیجه هزینه سرشکن:

(m + 1) + (2 - m) = 3

است.

بنابراین عملیاتی که در واقعیت ممکن است O(n) هزینه داشته باشد، از دید تحلیل سرشکن فقط یک هزینه ثابت دارد.

برای کاهش ظرفیت نیز همین اتفاق رخ می‌دهد. فرض کنیم پیش از حذف، تعداد عناصر برابر:

m / 4 + 1

باشد.

با حذف یک عنصر تعداد عناصر به:

$m / 4$

می‌رسد و ظرفیت از m به:

$m / 2$

کاهش پیدا می‌کند.

قبل از حذف، چون تعداد عناصر کمتر از نصف ظرفیت است، پتانسیل تقریباً برابر:

$m / 4 - 1$

است.

برای حذف یک عنصر و انتقال $m / 4$ عنصر باقی‌مانده، هزینه واقعی تقریباً:

$1 + m / 4$

خواهد بود.

پس از کاهش ظرفیت، تعداد عناصر دقیقاً نصف ظرفیت جدید است و پتانسیل جدید برابر صفر می‌شود.

در نتیجه هزینه سرشکن برابر می‌شود با:

$1 + m / 4 + 0 - (m / 4 - 1)$

که برابر است با:

2

بنابراین عملیات کاهش ظرفیت نیز با وجود هزینه واقعی زیاد، دارای هزینه سرشکن ثابت است.

در نتیجه نهایی، اگرچه یک عملیات خاص افزودن یا حذف می‌تواند در بدترین حالت O(n) باشد، به دلیل فاصله مناسب میان تغییرات ظرفیت و ذخیره شدن هزینه در عملیات‌های ساده، هزینه سرشکن هر عملیات برابر:

O(1)

خواهد بود.